

GitHub Actions Security in Python Packages

Andrew Nesbitt

PyCon US 2026, Long Beach

Andrew Nesbitt

- Package Manager Nerd
- Open Source Data Miner
- Dependency Graph Cartographer
- Critical Infrastructure Hobbyist
- Poodle Farmer



ecosyste.ms

- 14 million packages
- 157 million versions
- 300 million repositories
- 400 million manifest files
- 24 billion dependency edges

Open data and APIs for every major package registry.

nesbitt.io - my blog

- [Package Managers Need to Cool Down](#)
- [Sandwich Bill of Materials](#)
- [Incident Report: CVE-2024-YIKES](#)
- [If It Quacks Like a Package Manager](#)
- [How to Attract AI Bots to Your Open Source Project](#)
- [Weekend at Bernie's](#)

nesbitt.io - the relevant bits

- GitHub Actions Has a Package Manager, and It Might Be the Worst
- GitHub Actions is the Weakest Link

GitHub Actions

Let's talk about CI

GitHub Actions in PyPI

- 864,085 PyPI packages total
- 386,957 PyPI packages declare repository on GitHub
- 343,292 unique GitHub repositories
- 152,318 of those repositories have `.github/workflows/`
- Travis CI: 11%, others below 2%

Publishing to PyPI from GitHub Actions

- Tag → workflow runs → wheel built → `twine upload`
- 56,490 repos use `pypa/gh-action-pypi-publish`
- ~22% via OIDC, 44,181 still on a stored `PYPI_API_TOKEN`
- The runner that builds your wheel has your publish credential

Trusted publishing

- No stored API token
- PyPI accepts a short-lived OIDC token from the workflow
- Identity = repo + workflow filename + environment
- PyPI now trusts the workflow itself
- docs.pypi.org/trusted-publishers

Trusted publishing isn't enough

- Signing happens after the workflow runs
- Attestations (PEP 740, Sigstore, SLSA) verify the artifact came from the workflow
- They don't verify the workflow wasn't compromised first
- Signing happens last
- Everything upstream of the upload step is in scope

GitHub Actions is the Weakest Link

Let me list the ways

`uses:` is a dependency declaration

- `uses: actions/checkout@v4`
- `uses: astral-sh/setup-uv@v3`
- `uses: tj-actions/changed-files@v41`

- Pulls someone else's code and executes it on your runner
- Functionally, that's `pip install`
- Except `@v41` is a git ref, not an immutable version

Missing package-manager features

- No lockfile
- No hash verification
- No signature verification
- No transitive dependency resolution
- No yank / recall

Tags are mutable

```
git tag -f v41 <malicious-sha>  
git push -f origin v41
```

- Every workflow on `@v41` now runs the attacker's code
- Re-run last week's green build, get different code
- tj-actions, Trivy: exactly this

Transitive dependencies are not pinned

```
# your workflow, pinned
- uses: some/action@<sha>

# inside that action's action.yml
- uses: other/helper@main          # not pinned, you never see it
```

- No resolver, no tree, no `pip lock`
- reviewdog → tj-actions chain went through this

Nine compromises in eighteen months

Project	Date	Compromise
spotbugs	Nov 2024	<code>pull_request_target</code> ran fork code
Ultralytics	Dec 2024	cache poisoning → PyPI
tj-actions / reviewdog	Mar 2025	tags force-pushed, 23k repos
Trivy ×2	Mar 2026	75/76 tags force-pushed, CI token harvested
LiteLLM	Mar 2026	stolen PyPI token via Trivy chain → PyPI
Telnyx	Mar 2026	stolen PyPI token via Trivy chain → PyPI
elementary-data	Apr 2026	template injection → PyPI
lightning	Apr 2026	stale long-lived token, no OIDC → PyPI
Mini Shai-Hulud	May 2026	cache poison + OIDC theft → PyPI (mistralai, guardrails-ai)

Ultralytics, Dec 2024

fork PR → format.yml runs fork code (pull_request_target)

→ branch name interpolates into shell

→ poisons GitHub Actions cache

publish workflow → restores poisoned cache

→ wheel built with Crypto miner

→ uploaded to PyPI as 8.3.41, 8.3.42

phase 2 → stolen PYPI_TOKEN, direct upload of 8.3.45, 8.3.46

- dangerous-triggers, template-injection, cache-poisoning
- Template-injection bug reported and patched in August 2024, regression reintroduced ten days after the advisory
- **1,348 PyPI repos** have the same dangerous-triggers + cache-poisoning

How bad is it?

Using zizmor to find common misconfigurations



zizmor

```
$ uvx zizmor .github/workflows/
```

- Static analyser for Actions workflows
- Named audits, severity + confidence
- Runs locally or in CI
- [zizmor.sh](https://github.com/psf/black)

PyPI packages with workflows

- Every PyPI package with a linked GitHub repo, via ecosyste.ms
- Dependents + download counts for ranking
- **152,318** packages with `.github/workflows/`
- ~20% of linked repos fail to clone: package still installable, source gone

Running zizmor on everything

- Shallow clone each repo
- `zizmor --format=json .github/workflows/`
- Also extract every `uses:` line for an actions inventory
- Put everything into SQLite

github.com/andrew/pycon

Limits of static YAML analysis

- Workflow files only
- Each file considered separately
- Repo settings are out of scope: default token permissions, branch protection, etc
- A finding means the YAML permits the pattern, not always exploitable

Findings

Common misconfigurations across Python packages

Six audits

audit	PyPI repos	GHSA advisories
<code>excessive-permissions</code>	102,235	6
<code>unpinned-uses</code> *	85,774	4
<code>use-trusted-publishing</code>	44,181	n/a
<code>template-injection</code>	21,166	27
<code>cache-poisoning</code>	15,371	2
<code>dangerous-triggers</code>	7,025	8

* `unpinned-uses` *only for third-party actions*

excessive-permissions

```
on: push
jobs:
  build:           # no permissions: block
    steps: ...
```

- Without `permissions:`, the job inherits the repo default
- For repos created before Feb 2023: `contents: write`, `actions: write`, ...
- Any compromised step can push commits and dispatch workflows
- **102,235** pypi repos have at least one workflow without `permissions: {}`

unpinned-uses

```
- uses: tj-actions/changed-files@v41
```

- `@v41` is a git tag; the owner (or attacker) can move it
- Next run executes whatever the tag now points at
- **85,774** repos use a third-party action by tag
- **4** published compromises via this: tj-actions, reviewdog, Trivy, xygeni

template-injection

```
- run: echo "PR title: ${ { github.event.pull_request.title } }"
```

- `${ { }` expands before bash sees the script
- Attacker-controlled fields (PR title, branch name, issue body) become shell
- **21,166** PyPI repositories interpolate attacker-controlled fields into `run:`
- **27** of 49 published Actions CVEs are template-injection

use-trusted-publishing

```
- run: twine upload dist/*  
  env:  
    TWINE_PASSWORD: ${ secrets.PYPI_API_TOKEN }
```

- Long-lived token stored in GitHub secrets
- The most valuable target for other vectors
- **44,181** repos still on tokens (~78% of `gh-action-pypi-publish` users)
- Including `six` (896M/mo), `fsspec` (616M), `sqlalchemy` (335M)

cache-poisoning

```
- uses: actions/cache@v4      # in a release job
  with:
    key: pip-${{ hashFiles('requirements.txt') }}
```

- Cache is shared across workflows in a repo
- A low-privilege job writes a poisoned entry; the release job restores it
- **15,371** PyPI repositories use a cache action in a way that could get poisoned
- **2** public CVE advisories

dangerous-triggers

```
on: pull_request_target
jobs:
  test:
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${ github.event.pull_request.head.sha }
      - run: pip install -e . && pytest
```

- `pull_request_target` runs in the base repo with secrets
- Checking out the PR head runs the fork's code with those secrets
- **7,025** PyPI repositories use `pull_request_target`
- **8** public CVE advisories

archived-uses

```
- uses: actions/create-release@v1
```

- Uses an action that has been archived on GitHub, i.e. deprecated
- No future updates
- No security patches
- Might break if the action's runtime changes (e.g. deprecates Node 16)
- Might break if GitHub changes the API the action relies on
- **3,625** PyPI repos depend on an archived `archived-uses`

Most popular third-party actions

action	PyPI repos	unpinned
<code>pypa/gh-action-pypi-publish</code>	56,490	84.0%
<code>codecov/codecov-action</code>	20,651	91.8%
<code>astral-sh/setup-uv</code>	17,047	85.6%
<code>softprops/action-gh-release</code>	9,327	91.3%
<code>pypa/cibuildwheel</code>	2,650	91.7%

What goes on inside an action?

```
# action.yml (the visible tier)
runs:
  using: composite
  steps:
    - run: python -m cibuildwheel ${{ inputs.package-dir }}
```

- 2,650 PyPI publish workflows depend on it
- Self-audits with zizmor, one Low finding
- Fetches interpreters from **7 upstream hosts** at runtime, no hash pin
- Runtime fetches: CPython, PyPy, GraalPy, virtualenv, Node.js, nuget

Third-party actions in publish jobs

action	repos	unpinned
<code>astral-sh/setup-uv</code>	3,819	90.5%
<code>softprops/action-gh-release</code>	2,448	93.7%
<code>python-semantic-release/python-semantic-release</code>	451	87.0%
<code>snok/install-poetry</code>	381	95.9%
<code>salsify/action-detect-and-tag-new-version</code>	265	99.6%

One tag-hijack here runs with PyPI credentials across thousands of packages.

GitHub's 2026 roadmap

Lockfiles, hashes, verification

GitHub's 2026 roadmap

- Workflow dependency locking, transitive SHAs
- Policy controls on triggers (ban `pull_request_target`)
- Scoped secrets
- Egress firewall for runners
- *No committed ship dates yet*

github.blog/news-insights/product-news/whats-coming-to-our-github-actions-2026-security-roadmap/

What's missing?

- Malware detection
- Yanking and recalling
- CVE alerts
- Enforcement of policies

Practical hardening

Checklist and zizmor integration

Hardening checklist

1. Trusted publishing with an environment
2. `permissions: {}` on every workflow
3. Pin third-party actions to SHA: `zizmor --fix=all`
4. Never `${{ }}` into `run:`, pass through `env:`
5. Keep the publish job minimal
6. Run zizmor in CI

Trusted publishing with an environment

```
jobs:
  pypi-publish:
    environment: release          # ← required reviewers here
    permissions:
      id-token: write
    steps:
      - uses: actions/download-artifact@v4
      - uses: pypa/gh-action-pypi-publish@release/v1
```

- OIDC removes the stealable credential
- Bind the publisher on PyPI to the environment name

Adding zizmor to CI

```
name: zizmor
on: [push, pull_request]
permissions: {}
jobs:
  zizmor:
    runs-on: ubuntu-latest
    permissions:
      security-events: write
    steps:
      - uses: actions/checkout@v4
      - uses: zizmorcore/zizmor-action@v0
```

Findings show up in the PR and the Security tab.

Zero-finding examples

- `requests`, `pytest`, `stamina`, `flask`, `django`, `boto3`
- Their release workflows are good templates to copy

Critical-set findings over time

audit	6 - 11 Apr	28 Apr	11 May
unpinned - uses	7,446	6,320	6,406
artipacked	2,755	2,337	2,376
excessive - permissions	2,186	1,887	1,900

PyPI critical set: ~15% drop in three weeks after Trivy and elementary-data security events.

Take aways

- 🦶 GitHub Actions is full of footguns
- 🌈 Review your workflows with zizmor
- :spy: Be cautious of 3rd party actions
- 🔒 Set up Trusted Publishing
- Kill `pull_request_target` with 🔥

Thanks

Andrew Nesbitt

nesbitt.io · @andrewnez · ecosyste.ms

Data + scan tooling: github.com/andrew/pycon

Support zizmor

If you find zizmor useful, please consider supporting it:

- [Contributing.md](#)
- [GitHub Sponsors](#)
- [Thanks.dev](#)
- [ko-fi](#)

